Manual 1: Accessing and Using CESNET S3 Object Storage

Pollination Ecology Group (Handrkov) — GUI, CLI, R, and Python Workflows

2026-11-12

Table of contents

1. Connection Details & Credentials . . . 2
    Tip: Retrieving the Group S3 Keys via `kubectl` . . . 2
2. Which Method Should I Use? . . . 2
3. Method A: Graphical Point-and-Click (Windows / macOS) . . . 3
    Option 1: Cyberduck (Windows & macOS — Recommended) . . . 3
    Option 2: WinSCP (Windows) . . . 3
4. Method B: Mounting S3 as a Local Folder (`s3fs` on Linux / macOS) . . . 4
    Step 1: Install `s3fs` . . . 4
    Step 2: Save Your Credentials to `~/.passwd-s3fs` . . . 4
    Step 3: Mount the Bucket . . . 4
    Step 4: Unmounting (and Fixing "Transport endpoint is not connected") . . . 5
5. Method C: Fast Bulk Transfers & Backups (`rclone` & `aws-cli`) . . . 5
    Setting up `rclone` (One-Time Setup) . . . 5
    Everyday `rclone` Commands . . . 6
    Using `aws-cli` (Alternative CLI) . . . 6
6. Method D: Reading & Writing Directly from R and Python . . . 7
    Using CESNET S3 in R (`aws.s3`) . . . 7
    Using CESNET S3 in Python (`boto3`) . . . 8
7. House Rules for `peg-bucket` Organization . . . 9

CESNET provides high-capacity, S3-compatible object storage (powered by Ceph) where our group stores raw imagery, microscopy scans, ecological datasets (`.csv`, `.rds`), and trained deep-learning models.

Because S3 is an **object storage** system (accessible over HTTPS rather than a traditional local hard drive), you can connect to it from **any operating system (Windows, macOS, Linux)** as well as directly from **R**, **Python**, and **Kubernetes Pods**.

---

1. Connection Details & Credentials

To connect to our group storage with any tool, you need four pieces of information:

Setting	Value
Endpoint URL	`https://s3.cl4.du.cesnet.cz` *(Host: `s3.cl4.du.cesnet.cz`, Port: 443)*
Bucket Name	`peg-bucket` *(Shared group bucket)*
Region	Leave blank / empty (`""`) or `us-east-1` if a tool requires a dummy string
Addressing Style	**Path-style** (`https://s3.cl4.du.cesnet.cz/peg-bucket/...`)
Access Key & Secret Key	Obtain from Jakub / the group password manager *(or retrieve from Kubernetes if you already have cluster access—see tip below)*

> **Note**
>
> **Legacy Bucket Note:** Older projects prior to migration used endpoint `https://s3-old.cl4.du.cesnet.cz` with bucket name `bucket`. All active and new work should use `https://s3.cl4.du.cesnet.cz` and `peg-bucket`.

Tip: Retrieving the Group S3 Keys via `kubectl`

If you already have `kubectl` configured for our Kubernetes namespace (`stenc-ns`), you can print the shared S3 `Access Key` and `Secret Key` directly in your terminal:

```
# Print Access Key:
kubectl get secret s3-credentials -n stenc-ns -o jsonpath='{.data.access-key}' | base64 -d ; ech

# Print Secret Key:
kubectl get secret s3-credentials -n stenc-ns -o jsonpath='{.data.secret-key}' | base64 -d ; ech
```

---

2. Which Method Should I Use?

Choose the workflow that best matches your task:

1. **Method A: Graphical App (Cyberduck or WinSCP)** — Best for **Windows & macOS users** who want to browse folders and drag-and-drop files without using the terminal.
2. **Method B: Mount as a Local Folder (`s3fs`)** — Best for **Linux/macOS users** who want `peg-bucket` to appear as a regular directory (`~/cesnet_s3`) to inspect files or open small tables. *(Do not use `s3fs` for copying 10,000+ small files—use `rclone` instead!)*

3. **Method C: Fast Bulk Transfers & Backups (`rclone` / `aws-cli`)** — Best when uploading or downloading **gigabytes of images, microscopy `.czi` scans, or `.zip` datasets**. Runs 10x–50x faster than `s3fs` using parallel threads.
4. **Method D: Direct Access in R (`aws.s3`) or Python (`boto3`)** — Best for **reproducible data analysis scripts** so your code reads data straight from S3 and writes outputs back without manual file copying.

---

3. Method A: Graphical Point-and-Click (Windows / macOS)

Option 1: Cyberduck (Windows & macOS — Recommended)

1. Download and install Cyberduck (free and open-source).
2. Click **Open Connection** (top-left) and select **Amazon S3** from the top dropdown menu.
3. Fill in the connection dialog:
   - **Server:** `s3.cl4.du.cesnet.cz`
   - **Port:** `443`
   - **Access Key ID:** `<YOUR_ACCESS_KEY>`
   - **Secret Access Key:** `<YOUR_SECRET_KEY>`
4. Click **More Options** (at the bottom of the dialog) and set:
   - **Path:** `peg-bucket` *(entering `peg-bucket` directly here takes you straight into our bucket even if your key does not have permission to list all buckets on the server!)*
5. Click **Connect**. You can now browse folders, drag-and-drop files to upload/download, and bookmark the connection for one-click access.

Option 2: WinSCP (Windows)

1. Download and install WinSCP.
2. In the Login dialog, set:
   - **File protocol:** `Amazon S3`
   - **Host name:** `s3.cl4.du.cesnet.cz`
   - **Port number:** `443`
   - **Access key ID:** `<YOUR_ACCESS_KEY>`
   - **Secret access key:** `<YOUR_SECRET_KEY>`
3. Click **Advanced...** -> **Environment** -> **Directories** and set **Remote directory** to `/peg-bucket`.
4. Under **Environment** -> **S3**, set **URL style** to **Path** (`Path-style`).
5. Click **OK**, then **Save** the site profile, and click **Login**.

---

4. Method B: Mounting S3 as a Local Folder (`s3fs` on Linux / macOS)

With `s3fs`, you can mount `peg-bucket` as a folder in your home directory (e.g., `~/cesnet_s3`).

Step 1: Install `s3fs`

- **Ubuntu / Debian / Linux Mint:**

```
sudo apt update && sudo apt install -y s3fs
```

- **Fedora:**

```
sudo dnf install -y s3fs-fuse
```

- **macOS (via Homebrew):**

```
brew install --cask macfuse
brew install gromgit/fuse/s3fs-mac
```

Step 2: Save Your Credentials to `~/.passwd-s3fs`

Run the following two commands in your terminal (replace `<YOUR_ACCESS_KEY>` and `<YOUR_SECRET_KEY>` with the actual keys; notice the colon `:` between them and **no newline**):

```
printf '<YOUR_ACCESS_KEY>:<YOUR_SECRET_KEY>' > ~/.passwd-s3fs
chmod 600 ~/.passwd-s3fs
```

> **!** Important
>
> `s3fs` will refuse to run if `~/.passwd-s3fs` is readable by other users. `chmod 600 ~/.passwd-s3fs` is mandatory.

Step 3: Mount the Bucket

You can use our ready-made helper script `templates/s3/mount_cesnet_s3.sh`, or run the mount command directly:

```
mkdir -p ~/cesnet_s3
s3fs peg-bucket ~/cesnet_s3 \
  -o passwd_file=${HOME}/.passwd-s3fs \
  -o url=https://s3.cl4.du.cesnet.cz \
  -o use_path_request_style \
  -o uid=$(id -u) \
  -o gid=$(id -g)
```

Verify that your files are visible:

```
ls -la ~/cesnet_s3
```

Step 4: Unmounting (and Fixing "Transport endpoint is not connected")

When you are done, or if your laptop wakes up from sleep after a Wi-Fi disconnect and `~/cesnet_s3` freezes:

```
fusermount -u ~/cesnet_s3
# Or if busy/hung:
fusermount -uz ~/cesnet_s3
```

Then run the `s3fs` mount command again.

---

5. Method C: Fast Bulk Transfers & Backups (`rclone` & `aws-cli`)

> **Tip**
>
> **Why not use `s3fs` for large transfers?** FUSE mounts (`s3fs`) process files one-by-one and make multiple metadata calls per file. If you are uploading 5,000 pollinator images or a 20 GB dataset, **`rclone` is 10x to 50x faster** because it transfers 16–32 files in parallel and verifies checksums automatically.

Setting up `rclone` (One-Time Setup)

1. **Install `rclone`:**
   - **Linux:** `sudo apt install -y rclone`
   - **macOS:** `brew install rclone`
   - **Windows:** `winget install Rclone.Rclone` (or download from rclone.org)
2. **Create the configuration file (`~/.config/rclone/rclone.conf`):** Run `mkdir -p ~/.config/rclone` and add the following block to `~/.config/rclone/rclone.conf`:

```
[cesnet-s3]
type = s3
provider = Ceph
access_key_id = <YOUR_ACCESS_KEY>
secret_access_key = <YOUR_SECRET_KEY>
endpoint = https://s3.cl4.du.cesnet.cz
acl = private
```

Everyday `rclone` Commands

```
# 1. List top-level folders inside peg-bucket
rclone lsd cesnet-s3:peg-bucket

# 2. List directories inside Ostatni/
rclone lsd cesnet-s3:peg-bucket/Ostatni/

# 3. Copy a local folder TO CESNET S3 (High-speed parallel upload)
rclone copy ~/MyProjectData/ cesnet-s3:peg-bucket/Ostatni/MyProjectData/ \
  --update --transfers 16 --checkers 16 --fast-list --progress

# 4. Download a folder FROM CESNET S3 to your computer
rclone copy cesnet-s3:peg-bucket/Ostatni/MyProjectData/ ~/MyProjectData/ \
  --update --transfers 16 --checkers 16 --fast-list --progress
```

> **Warning**
>
> **CRITICAL SAFETY RULE: Always use `rclone copy`, NOT `rclone sync`!**
>
> - `rclone copy` only adds new or updated files to the destination—it **never deletes** files on S3.
> - `rclone sync` makes the destination identical to the source, meaning if you accidentally point an empty local folder at `peg-bucket`, `rclone sync` would delete files from S3! Always use `rclone copy` (or test first with `--dry-run`).

See `templates/s3/rclone_backup_template.sh` for a complete automated backup script with logging.

Using `aws-cli` (Alternative CLI)

If you prefer the standard Amazon S3 CLI (`pip install awscli`), always pass `--endpoint-url https://s3.cl4.du.cesnet.cz`:

```
export AWS_ACCESS_KEY_ID="<YOUR_ACCESS_KEY>"
export AWS_SECRET_ACCESS_KEY="<YOUR_SECRET_KEY>"

# List contents of peg-bucket
aws --endpoint-url https://s3.cl4.du.cesnet.cz s3 ls s3://peg-bucket/Ostatni/

# Copy a single file to S3
aws --endpoint-url https://s3.cl4.du.cesnet.cz s3 cp results.csv s3://peg-bucket/Ostatni/MyProje

# Sync a folder
aws --endpoint-url https://s3.cl4.du.cesnet.cz s3 sync ./my_plots/ s3://peg-bucket/Ostatni/MyPro
```

---

6. Method D: Reading & Writing Directly from R and Python

Writing scripts that read and write directly to S3 makes your code work identically on your laptop and inside a CESNET Kubernetes Job.

Using CESNET S3 in R (`aws.s3`)

1. **Store your credentials in `~/.Renviron`** so you never hardcode keys inside `.R` scripts: In R, run `usethis::edit_r_environ()` (or open `~/.Renviron` in any text editor) and add:

```
AWS_ACCESS_KEY_ID="YOUR_ACCESS_KEY"
AWS_SECRET_ACCESS_KEY="YOUR_SECRET_KEY"
AWS_S3_ENDPOINT="s3.cl4.du.cesnet.cz"
AWS_DEFAULT_REGION=""
```

   Restart R after saving `~/.Renviron`.

2. **Use `aws.s3` in your R script** (full template at `templates/s3/s3_example.R`):

```r
library(aws.s3)

# IMPORTANT: Always set region = "" and base_url = "s3.cl4.du.cesnet.cz"
# Otherwise aws.s3 tries to connect to Amazon's us-east-1 servers!
S3_BUCKET <- "peg-bucket"
S3_HOST   <- "s3.cl4.du.cesnet.cz"

# 1. List files in a folder
files_df <- get_bucket_df(
  bucket = S3_BUCKET,
  prefix = "Ostatni/",
  base_url = S3_HOST,
  region = "",
  use_https = TRUE
)
head(files_df[, c("Key", "Size", "LastModified")])

# 2. Read an .rds file directly from S3 into R memory
my_data <- s3readRDS(
  object = "Ostatni/MyProject/cleaned_data.rds",
  bucket = S3_BUCKET,
  base_url = S3_HOST,
  region = "",
  use_https = TRUE
)

# 3. Save an R object directly to S3 as an .rds file
s3saveRDS(
  x = my_data,
  object = "Ostatni/MyProject/model_output.rds",
  bucket = S3_BUCKET,
```

```
  base_url = S3_HOST,
  region = "",
  use_https = TRUE
)

# 4. Upload any local file (e.g. PDF plot or CSV) to S3
put_object(
  file = "figure1.pdf",
  object = "Ostatni/MyProject/figures/figure1.pdf",
  bucket = S3_BUCKET,
  base_url = S3_HOST,
  region = "",
  use_https = TRUE
)
```

Using CESNET S3 in Python (`boto3`)

Install `boto3` (`pip install boto3 pandas`) and use the helper pattern below (full template at `templates/s3/s3_example.py`):

```
import io
import os
import boto3
from botocore.config import Config
import pandas as pd

S3_ENDPOINT = os.environ.get("S3_ENDPOINT", "https://s3.cl4.du.cesnet.cz")
S3_BUCKET   = os.environ.get("S3_BUCKET", "peg-bucket")

s3 = boto3.client(
    "s3",
    endpoint_url=S3_ENDPOINT,
    aws_access_key_id=os.environ["AWS_ACCESS_KEY_ID"],
    aws_secret_access_key=os.environ["AWS_SECRET_ACCESS_KEY"],
    config=Config(signature_version="s3v4"),
)

# 1. Read a CSV directly from S3 into a pandas DataFrame
obj = s3.get_object(Bucket=S3_BUCKET, Key="Ostatni/MyProject/data.csv")
df = pd.read_csv(io.BytesIO(obj["Body"].read()))

# 2. Download a file (e.g. a .zip dataset or .pt model weights)
s3.download_file(S3_BUCKET, "Ostatni/MyProject/dataset.zip", "dataset.zip")

# 3. Upload a local file to S3
s3.upload_file("results.csv", S3_BUCKET, "Ostatni/MyProject/results.csv")
```

7. House Rules for `peg-bucket` Organization

To keep our shared bucket clean and easy to navigate for everyone:

1. **Project Folders:** Place project-specific datasets under `Ostatni/<ProjectName>/` (e.g., `Ostatni/Stomata/`, `Ostatni/Pollen_viability/`, `Ostatni/YOLO_flies/`, `Ostatni/RhizoPic/`) or your dedicated top-level project directory.
2. **Avoid Millions of Tiny Loose Files:** When uploading training datasets with >10,000 small cropped images for Kubernetes GPU jobs, **pack them into a single `.zip` or `.tar` file** in a `staged_area/` subfolder too. Downloading a single 2 GB `.zip` file inside a Kubernetes Job takes 5 seconds; downloading 20,000 tiny `.jpg` files individually takes 15 minutes!
3. **Never Run `rclone sync` or `aws s3 rm --recursive` Without Checking Path Twice:** Double-check your target prefix before deleting or overwriting data.